

Electro-Concept Mauricie

TPlusNavigator documentation



April 2004.

TPlusNavigator documentation

Table of Contents

OVERVIEW	1
TPLUSNAVIGATOR DESCRIPTION	1
SOURCE CODE FILES	1
PURPOSE OF THIS DOCUMENT	1
MANUAL CONVENTIONS.....	2
PROGRAMMING INTERFACE	3
BASICS OF TPLUSNAVIGATOR USAGE.	3
POSITION SETTING AND RETRIEVAL PROPERTIES.	3
TEXT CHARACTERISTICS PROPERTIES.	5
METHODS.....	5
<i>INDEX</i>.....	1

Overview

TPlusNavigator description

TPlusNavigator is a Delphi class developed specially to work with the TPlusMemo component from *Electro-Concept Mauricie* (ECM). It is used to “navigate” in the text of the component and to query the text characteristics.

For example, it is possible to position a *TPlusNavigator* by specifying its offset from start, then query some of its properties to know its particular paragraph number, line number, display position, etc. The position can be specified in terms of offset from start, paragraph number, line number, or column number. It is also possible to determine the font characteristics at a given position, and the dynamic highlighting attributes.

One important aspect of *TPlusNavigator* objects is that once they are set to a given position, they will automatically track the text at that position. It means that if some text is added or deleted before it, the position properties will be updated to reflect the new position of the same text.

Internally, TPlusMemo uses a number of *TPlusNavigator*'s for its working. These should be considered proprietary and must not be repositioned or destroyed. They can be used to query some particular aspect of the state of a TPlusMemo. For example, property `MouseNav` is a *TPlusNavigator* instance that tracks the mouse pointer location. Similarly, `SelStartNav` and `SelStopNav` are *TPlusNavigator*'s that constantly represent the selection boundaries. Also, many PlusMemo accessories use *TPlusNavigator*'s for their internal working.

Source code files

The source code for *TPlusNavigator* is contained in unit `PMSupport.pas`. Adding unit `PMSupport` to your uses clause may be required to work with *TPlusNavigator*'s in your application.

Purpose of this document

This document has been written for the purpose of helping application programmers achieve special effects with the TPlusMemo component through the use of *TPlusNavigator* objects. We recommend that programmers limit the usage of *TPlusNavigator* objects to properties and methods exposed here. Undocumented properties and methods are also present but their purpose is for TPlusMemo internal working and should not be used. Their programming interface may also change with future releases of TPlusMemo.

Manual conventions

The text attributes in the following sections are set according to these three conventions:

- Italicized parts refer to a topic introduced elsewhere in this document. Most of these topics have an entry in the index at the end.
- Underlined parts refer to a topic related to the TPlusMemo component and documented in the PlusMemo.hlp help file.
- Italicized and underlined parts means that the topic is covered in the Delphi help system.

Furthermore, unless otherwise noted, this document applies equally to *TPlusNavigatorU* objects, which is the Unicode version of *TPlusNavigator* for use with TPlusMemoU components.

Programming interface

Basics of TPlusNavigator usage.

Programmers who want to use a *TPlusNavigator* object first need to create one by calling its special *Create* constructor, which takes a parameter of type TPlusMemo. Once created, the *TPlusNavigator* object is considered to be “attached” to the TPlusMemo instance passed in the *Create* call.

After creation, programmers typically use various properties and methods of *TPlusNavigator* to achieve the effect they want. When it is not needed anymore, the *TPlusNavigator* object can be freed by calling its *Free* method. If a *TPlusNavigator* is not freed at the time the TPlusMemo to which it is attached is destroyed, the TPlusMemo will free it, so the application programmer must be aware that every references to said *TPlusNavigator* will be invalid after the TPlusMemo is freed.

The lifetime of a *TPlusNavigator* can be as short as the duration of an event handler, or as long as the life of the TPlusMemo to which it is attached. The latter circumstances are typically encountered when a *TPlusNavigator* is used to keep track of some text position while editing operations are done by the user or by application code. The former situation, where the *TPlusNavigator* is created and destroyed in the same scope, is typically used for querying the text characteristics at a given time when the application runs.

There is no hard limit on the number of *TPlusNavigator* that can be used at the same time, but each one attached to a given TPlusMemo imposes a load on it. For this reason, programmers should limit the number of *TPlusNavigator*’s that are alive at the same time to the minimum value required to achieve the desired task. Still, it has been verified that a fairly large amount of them can be supported: in practice, no performance degradation was observed for as many as two thousand navigators attached at the same time to a TPlusMemo, for a Pentium class system.

Position setting and retrieval properties.

The following properties are used to position a *TPlusNavigator* or to query one characteristic related to a particular position. They all have read and write access. Upon writing to one of them, the others are automatically updated.

In the description that follows, PlusMemo refers to the instance of TPlusMemo that the *TPlusNavigator* is “attached” to.

Pos: Longint;

This property corresponds to the offset from start. A value of zero represents the start of the text. Each end of paragraph counts for two characters. This value is not range checked, so the programmer should not assign it a value outside the range of 0 to PlusMemo.CharCount. Otherwise, it is likely that errors will occur when working with the *TPlusNavigator* afterward. Also, care must be taken to ensure that a *TPlusNavigator* is not

positioned mid-way between the two characters that represent a paragraph delimiter (#13#10). The behavior is not specified in this circumstance.

ParNumber: Longint;

This property indicates the paragraph number of the current position. A value of zero means the first paragraph. Upon assigning to it, the *TPlusNavigator* will go to the first character of the paragraph indicated by *ParNumber*. Attempting to assign to it a value outside the range of 0 to PlusMemo.ParagraphCount-1 will produce a run time error.

LineNumber: Longint;

This property indicates the line number of the current position. A value of zero corresponds to the first line. Upon assigning a value to this property, the *TPlusNavigator* will go to the first character of the corresponding line. Assignments to this property should be restricted to the range of 0 to PlusMemo.LineCount-1, otherwise a run time error will occur.

Col: Integer;

This property corresponds the column number of the current position. A value of zero means the first character of the current line. Setting it to a value greater than the current line length will make the *TPlusNavigator* go to the last position of the current line. Assigning a value lower than zero to property *Col* results in unspecified behavior.

ParOffset: Integer;

This property corresponds to the offset relative to the start of the current paragraph. A value of zero corresponds to the start of a paragraph. Setting it to a value greater than the length of the current paragraph will make the *TPlusNavigator* go to the last position of this paragraph (i.e. just before the virtual paragraph delimiter). Assigning to *ParOffset* a value smaller than zero must not be done.

ParLine: Integer;

This property corresponds to the line number of the current position, relative to the start of the current paragraph. Setting it to a value greater or equal than the number of lines in the current paragraph will make the *TPlusNavigator* move to the last line of it. Assigning a value smaller than zero must be avoided.

DisplayPos: TPoint;

This property corresponds to the display position of the *TPlusNavigator* object, in terms of client coordinates. When reading it, it will return a TPoint record where X and Y are respectively the horizontal and vertical position where the character pointed to by the navigator is displayed. When writing to it, the *TPlusNavigator* will move to the character present at that position. The coordinates are brought within range before this operation.

Text characteristics properties.

TPlusNavigator defines a few read only properties that let the programmer determine the characteristics of the text at its current position:

Text: Char; (WideChar for *TPlusNavigatorU*);

This property returns the character at the current position.

Style: TFontStyles;

This property returns the static character style at the current position. It is not affected by dynamic highlighting.

Context: SmallInt;

This property returns the context number corresponding to the dynamic highlighting keyword or start-stop key at the current position. If the current position is not under dynamic highlighting effect, property Context returns zero.

Methods.

constructor Create(*APlusMemo*: TComponent);

This constructor is used to create a *TPlusNavigator* object and attach it to a TPlusMemo component. Although the *APlusMemo* parameter is specified to be of type TComponent, the constructor will raise an exception at run time if the passed *APlusMemo* is not of type TPlusMemo.

procedure Assign(Source: *TPlusNavigator*);

This method is used to set all position properties equal to those of Source. Source navigator must be attached to the same TPlusMemo component as the current one, otherwise errors can occur.

function BackToDyn(Min: Longint): Boolean;

This method is used to go back to the previous dynamic highlighting boundary, if it is found to be at a location between *Min* (in offset from start of text) and the current position. If the previous dynamic highlighting boundary occurs between offset *Min* and the current position, the *TPlusNavigator* is moved there and the function result is True. If no dynamic highlighting boundary exists in the scanned range from *Min* to current position, the *TPlusNavigator* is not moved and the function result is False.

function ForwardToDyn(Max: Longint): Boolean;

This method is used to go to the next dynamic highlighting boundary, if it is found before an offset corresponding to parameter *Max*. If so, the *TPlusNavigator* is moved there and the function result is True. If not, the function result is False and no movement occurs.

procedure GetTextBuf(Buffer: PChar; Len: Integer);

This method is used to go get the text content starting from the current position, up to Len characters. Parameter Buffer must point to an already allocated PChar that is at least *Len*+1 long in size. A null character will be

appended to the character string. In the case of *TPlusNavigatorU*, parameter *Buffer* is of type *PWideChar*.

procedure *RightOfDyn*; ;

This method is used force the *TPlusNavigator* to take the dynamic attributes Normally, after moving a *TPlusNavigator* object, if it is set at a position corresponding to a dynamic highlighting boundary, it will take the dynamic attributes found at the left of this position. If you want to query the dynamic attributes corresponding to the right of this boundary without moving the *TPlusNavigator*, you can call *RightOfDyn*. Subsequent reading of property *Context* will return the value found after this boundary, contrarily to the value before this boundary if *RightOfDyn* is not called. If the current position is not a dynamic highlighting boundary, calling *RightOfDyn* has no effect.

INDEX

<i>A</i>	
Assign.....	5
<i>B</i>	
BackToDyn	5
<i>C</i>	
Col	4
Context	4
Create	5
<i>F</i>	
ForwardToDyn	5
<i>G</i>	
GetTextBuf.....	5
<i>L</i>	
LineNumber.....	4
<i>P</i>	
ParLine	4
ParNumber.....	4
ParOffset.....	4
Pos	3
<i>R</i>	
RightOfDyn	5
<i>S</i>	
Style.....	4
<i>T</i>	
Text.....	4
<i>TPlusNavigator</i>	1